

1. 프로그램 개요

AVR(16 MHz)로 HUB75 LED 패널 64×32 두 장을 세로로 체인하여 가상 64×64 화면을 만들고, 그 위에 2×스케일 테트리스를 표시한다. 게임 루프는 입력(오토리핏), 자동 낙하, 충돌/고정, 라인 삭제, 스폰을 처리하고, 렌더는 행쌍 스캔 + 2비트 PWM으로 패널을 구동한다. 점수에 따라 작은 춤(100점마다), STAGE CLEAR(300점), GAME/END 오버레이가 나온다.

2. 소프트웨어

1) 환경/헤더

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdint.h>
#include <avr/pgmspace.h>
```

- delay_us/ms()는 F_CPU를 기준으로 NOP 타이밍을 잡는다. 16 MHz에서 1us 단위 딜레이가 안전하게 나온다.
- pgmspace.h는 PROGMEM(플래시)에 올린 큰 테이블(폰트/조각) 읽을 때 필요. RAM 절약 핵심.

2) 패널/가상 해상도 상수

```
#define PANEL_W 64
#define PANEL_H 32
#define CHAIN_N 2
#define SHIFT_W (PANEL_W * CHAIN_N)
#define VIRT_W 64
#define VIRT_H 64
```

- 실제 64×32 패널 2장을 “세로 체인”해서 가상 화면 64×64을 만든다.
- 하지만 데이터는 가로로 연속 시프트해야 하므로 한 줄에 128픽셀을 밀어 넣는 셈 → SHIFT_W=128.
- 가상 좌표는 (vx, vy)=(0..63, 0..63). 이후 모든 렌더는 이 좌표로 계산한 뒤, 패널 물리에 맞춰 매핑한다.

3) 게임 레이아웃(픽셀/칸 좌표 연결)

```
#define FIELD_W 10
#define FIELD_H 32
#define BLOCK_SZ 2
#define PLAY_X0 12
#define SCORE_X0 40
#define SCORE_Y0 4
```

- 게임 칸→픽셀 스케일은 2×. 즉, (칸X,칸Y) = (픽셀X/2, 픽셀Y/2).
- 세로 32칸 × 2픽셀 = 64px로 가상 세로 전체를 꽉 채운다.
- 필드는 화면 왼쪽에서 12px 띄워 배치(테두리/점수 공간 확보). 점수 박스는 (40,4)에 고정.

4) 핀 매핑/토글/버튼

```
// ----- 핀 -----
#define R1_PD PD2           // 상단 R
#define G1_PD PD3
#define B1_PD PD4
#define R2_PD PD5           // 하단 R
#define G2_PD PD6
#define B2_PD PD7
#define CLK_PB PB0          // 시프트 클럭
#define OE_PB PB1           // 출력 인에이블
#define LAT_PB PB2          // 래치
#define A_PC PC0            // 행 선택 A~D
#define B_PC PC1
#define C_PC PC2
#define D_PC PC3
#define BTN_LEFT_PB PB3     // 좌/하 버튼
#define BTN_DOWN_PB PB4
#define BTN_RIGHT_PC PC4    // 우/회전 버튼
#define BTN_ROT_PC PC5

// 빠른 핀 토글 매크로
#define CLK_HIGH() (PORTB|=(1<<CLK_PB))
#define CLK_LOW() (PORTB&=~(1<<CLK_PB))
#define LAT_HIGH() (PORTB|=(1<<LAT_PB))
#define LAT_LOW() (PORTB&=~(1<<LAT_PB))
#define OE_HIGH() (PORTB|=(1<<OE_PB))
#define OE_LOW() (PORTB&=~(1<<OE_PB))

// 풀업 입력(활성 Low) 버튼 판독
static inline uint8_t btn_left(void){ return !(PINB&(1<<BTN_LEFT_PB)); }
static inline uint8_t btn_down(void){ return !(PINB&(1<<BTN_DOWN_PB)); }
static inline uint8_t btn_right(void){ return !(PINC&(1<<BTN_RIGHT_PC)); }
static inline uint8_t btn_rot(void){ return !(PINC&(1<<BTN_ROT_PC)); }
```

- HUB75는 CLK→LAT→OE 순으로 구동. 이 매크로는 오버헤드 없이 핀을 바꿔 PWM 타이밍을 지킨다.

- 버튼은 내부 풀업 활성화 → 평소 1, 눌림 0. 읽을 때 !로 뒤집어 “눌림=1”로 맞춘다(사용처 일관성).

5) 색상 팔레트(6비트 RGB)

```
// ----- 색 -----
// 2비트 RGB(각 0~3) 조합을 6비트 한 바이트로 압축
#define RGB6(r,g,b) ((uint8_t)((r)|((g)<<2)|((b)<<4)))
#define COL_BLACK      RGB6(0,0,0)
#define COL_BG1        RGB6(0,1,1)    // 어두운 남청
#define COL_BG2        RGB6(0,0,0)    // 블랙
#define COL_BORDER_DIM  RGB6(0,0,2)
#define COL_BORDER_BRT  RGB6(0,0,3)
#define COL_SCOREBOX    RGB6(1,1,1)
#define COL_I           RGB6(0,3,3)    // 시안
#define COL_O           RGB6(3,3,0)    // 노랑
#define COL_T           RGB6(3,0,3)    // 보라
#define COL_S           RGB6(3,1,1)    // 핑크(초록 대신)
#define COL_Z           RGB6(3,0,0)    // 빨강
#define COL_J           RGB6(3,3,3)    // 흰색(파랑 대신)
#define COL_L           RGB6(3,2,0)    // 주황
```

- 각 채널 2비트(0~3) → 한 바이트에 R(하위2) G(중간2) B(상위2)로 포장.
- 컬러 결정은 후반 get_pixel_color_virt()에서 하며, 마지막에 다시 R/G/B 2비트로 쪼개 PWM에 씬.

6) 상태 변수(게임/연출/속도)

```
// ----- 상태 -----
static uint8_t field[FIELD_H][FIELD_W]; // 맵: 0=빈 칸, 1~7=블록 색/종류
static int8_t cur_x, cur_y;             // 현재 조각 좌표(칸 단위)
static uint8_t cur_type, cur_rot;       // 현재 조각 타입/회전
static uint8_t rnd_state = 1;           // 간단 LFSR PRNG 상태

static uint16_t score = 0;               // 점수(0~999)
static uint8_t score_dig[3];            // 각 자리수 캐시

static uint16_t frame_cnt = 0;           // 낙하 타이머
static uint8_t drop_frames = 18;        // 자동 낙하 프레임 간격
#define DROP_FRAMES_MIN 8
#define SOFT_DROP_STEPS 2                // 소프트드롭 당 하강 칸

static uint8_t game_over = 0;           // 천장 도달 → 게임 종료 플래그

// 100점마다: 귀여운 12x12 버블 춤
static uint16_t dance_small_timer = 0;
#define DANCE_SMALL_DURATION 200
#define DANCE_SMALL_X0 44
#define DANCE_SMALL_Y0 18
#define DANCE_SMALL_COL1 RGB6(3,2,0)
#define DANCE_SMALL_COL2 RGB6(3,0,3)

// 300점: 스테이지 클리어 연출(게임 정지)
static uint8_t stage_clear_active = 0;
static uint16_t stage_clear_timer = 0;
#define STAGE_CLEAR_DURATION 360
```

- 필드는 칸 기준 10×32. 픽셀에서 그릴 때는 2× 스케일로 바뀌서 표시한다.
- drop_frames는 난이도. 프레임을 셀 때 이 값에 도달하면 한 칸 떨어뜨린다. 점수로 조절.
- dance_small_timer는 100점단위가 올라갈 때만 200으로 세트되어 자연감 있게 깜빡.
- stage_clear_active가 켜지면 게임 로직을 잠시 정지하고 오버레이만 그린다.

7) 폰트/문자 인덱스(요약)

```
// ----- 폰트 -----
// 3x5 숫자 비트맵(세로 5줄)
const uint8_t digit_font[10][5] PROGMEM = {
    {0x07,0x05,0x05,0x05,0x07},{0x02,0x06,0x02,0x02,0x07},{0x07,0x01,0x07,0x04,0x07},
    {0x07,0x01,0x07,0x01,0x07},{0x05,0x05,0x07,0x01,0x01},{0x07,0x04,0x07,0x01,0x07},
    {0x07,0x04,0x07,0x05,0x07},{0x07,0x01,0x01,0x01,0x01},{0x07,0x05,0x07,0x05,0x07},{0x07,0x05,0x07,0x01,0x07}
};

// 5x7 대문자 폰트 인덱스 및 데이터(필요한 문자만)
enum {CH_SPC=0, CH_A, CH_C, CH_D, CH_E, CH_G, CH_L, CH_N, CH_R, CH_S, CH_T};
const uint8_t font5x7[][7] PROGMEM = {
    {0,0,0,0,0,0,0}, // space
    {0x1C,0x22,0x22,0x3E,0x22,0x22,0x22}, // A
    {0x3C,0x22,0x22,0x3C,0x22,0x22,0x3C}, // C
    {0x3C,0x22,0x22,0x22,0x22,0x22,0x3C}, // D
    {0x3E,0x20,0x20,0x3C,0x20,0x20,0x3E}, // E
    {0x1E,0x20,0x20,0x2E,0x22,0x22,0x1E}, // G
    {0x22,0x22,0x22,0x22,0x22,0x22,0x3E}, // L
    {0x22,0x32,0x2A,0x26,0x22,0x22,0x22}, // N
    {0x3C,0x22,0x22,0x3C,0x28,0x24,0x22}, // R
    {0x1E,0x20,0x20,0x1C,0x02,0x02,0x3C}, // S
    {0x3E,0x08,0x08,0x08,0x08,0x08,0x08} // T
};

static inline uint8_t fidx(char c){ // 문자+폰트 인덱스
    switch(c){
        case 'A':return CH_A; case 'C':return CH_C; case 'D':return CH_D; case 'E':return CH_E;
        case 'G':return CH_G; case 'L':return CH_L; case 'N':return CH_N; case 'R':return CH_R;
        case 'S':return CH_S; case 'T':return CH_T; default: return CH_SPC;
    }
}
```

- 점수 표시는 3×5, 텍스트(STAGE/CLEAR/GAME/END)는 5×7.
- PROGMEM에 넣어 RAM 절약. fidx()는 문자→폰트 인덱스 매핑. 지원하지 않는 문자는 공백.

8) 테트로미노(7종×4회전×4줄)

```
// ----- 테트로미노(4x4) -----
// 각 조각별 회전 상태(0~3) x 로컬 4행 비트마스크(왼쪽이 MSB)
const uint8_t tetromino[7][4][4] PROGMEM = {
    {{0x00,0x00,0x00,0x0F},{0x02,0x02,0x02,0x02},{0x00,0x00,0x00,0x0F},{0x02,0x02,0x02,0x02}},
    {{0x00,0x06,0x06,0x00},{0x00,0x06,0x06,0x00},{0x00,0x06,0x06,0x00},{0x00,0x06,0x06,0x00}},
    {{0x00,0x0E,0x04,0x00},{0x04,0x06,0x04,0x00},{0x04,0x0E,0x00,0x00},{0x04,0x0C,0x04,0x00}},
    {{0x00,0x06,0x0C,0x00},{0x08,0x0C,0x04,0x00},{0x00,0x06,0x0C,0x00},{0x08,0x0C,0x04,0x00}},
    {{0x00,0x0C,0x06,0x00},{0x04,0x0C,0x08,0x00},{0x00,0x0C,0x06,0x00},{0x04,0x0C,0x08,0x00}},
    {{0x00,0x0E,0x02,0x00},{0x06,0x04,0x04,0x00},{0x08,0x0E,0x00,0x00},{0x04,0x04,0x0C,0x00}},
    {{0x00,0x0E,0x08,0x00},{0x04,0x04,0x06,0x00},{0x02,0x0E,0x00,0x00},{0x0C,0x04,0x04,0x00}}
};
```

- 4×4 로컬 좌표에서 어느 칸이 찢는지 비트마스크로 정의.
- 회전 상태(0~3)별로 미리 저장해서 회전 시 배열 인덱스만 바꿔 읽는다

9) RNG(LFSR)

```
// ----- RNG -----  
// 8비트 LFSR(다항식 0xB8)로 간단 난수 생성  
static uint8_t rand8(void){  
    uint8_t l=rnd_state;  
    l = (l&1)?((l>>1)^0xB8):(l>>1);  
    rnd_state=l; return l;  
}
```

- 8-bit LFSR(다항식 0xB8). 테트리스 “7백”은 아니지만, 임베디드에 충분히 가볍다.
- 스폰 시 rand8()%7로 조각 선택.

10) 유틸 함수들

```
// ----- 유틸 -----  
static uint8_t block_color(uint8_t t){ // 조각 인덱스→색상  
    switch(t){  
        case 1:return COL_I; case 2:return COL_O; case 3:return COL_T;  
        case 4:return COL_S; case 5:return COL_Z; case 6:return COL_J;  
        case 7:return COL_L; default:return COL_BLACK;  
    }  
}
```

- t는 1~7(필드에 저장된 값). 색 팔레트에서 즉시 조회.

```
static void update_score_digits(void){ // 점수 숫자 분해 캐시  
    uint16_t s=score; if(s>999)s=999;  
    score_dig[2]=s%10; s/=10;  
    score_dig[1]=s%10; s/=10;  
    score_dig[0]=s%10;  
}
```

- 숫자 렌더링이 빈번하니 자리수를 캐시. 렌더 시 분해 비용 0.

```
static void clear_field(void){ // 필드/상태 초기화  
    for(uint8_t y=0;y<FIELD_H;y++) for(uint8_t x=0;x<FIELD_W;x++) field[y][x]=0;  
    score=0; update_score_digits();  
    game_over=0; stage_clear_active=0; stage_clear_timer=0; dance_small_timer=0;  
}
```

- 재시작 경로에서 호출. 필드/점수/이펙트/상태를 짝 초기화한다.

11) 조각 처리(핵심)

```
// ----- 조각 로직 -----
// 이동/회전 가능 여부 검사
static uint8_t can_move(int8_t dx, int8_t dy, int8_t drot){
    uint8_t nr=(cur_rot+drot)&3;          // 목표 회전
    for(uint8_t ly=0; ly<4; ly++){
        uint8_t row=pgm_read_byte(&tetromino[cur_type][nr][ly]);
        if(!row) continue;
        for(uint8_t lx=0; lx<4; lx++){
            if(!(row&(1<<(3-lx)))) continue;          // 비어있으면 스킵
            int16_t nx=(int16_t)cur_x+dx+lx;
            int16_t ny=(int16_t)cur_y+dy+ly;
            if(nx<0||nx>=FIELD_W) return 0;          // 좌우 벽 충돌
            if(ny<0) continue;          // 위쪽은 허용
            if(ny>=FIELD_H) return 0;          // 바닥 충돌
            if(field[ny][nx]) return 0;          // 고정 블록 충돌
        }
    }
    return 1;
}
```

- (dx,dy,drot)만큼 이동/회전한 가정 상태에서 4×4를 스캔.
- ny<0은 충돌로 보지 않음: 스폰 직후 회전/이동 여지 제공(천장 위 공간).
- 단, 바닥(ny>=FIELD_H)과 좌우 벽, 고정 블록은 즉시 실패.
- 암묵적 불변식: cur_x,cur_y는 “왼쪽-위” 기준. 4×4 범위를 벗어나더라도 내부에서 좌표를 따로 검사하므로 안전.

```
// 현재 조각 고정 + 가득 찬 줄 삭제 + 점수
static void lock_and_clear(void){
    // 현재 조각을 필드에 써넣기
    for(uint8_t ly=0; ly<4; ly++){
        uint8_t row=pgm_read_byte(&tetromino[cur_type][cur_rot][ly]);
        if(!row) continue;
        for(uint8_t lx=0; lx<4; lx++){
            if(!(row&(1<<(3-lx)))) continue;
            int16_t gx=(int16_t)cur_x+lx;
            int16_t gy=(int16_t)cur_y+ly;
            if(gx<0||gx>=FIELD_W||gy<0||gy>=FIELD_H) continue;
            field[gy][gx]=cur_type+1;
        }
    }
    // 라인 클리어
    uint8_t lines=0;
    for(int8_t y=FIELD_H-1; y>=0; y--){
        uint8_t full=1; for(uint8_t x=0; x<FIELD_W; x++){ if(!field[y][x]){full=0;break;} }
        if(full){
            for(int8_t yy=y; yy>0; yy--) for(uint8_t x=0; x<FIELD_W; x++) field[yy][x]=field[yy-1][x];
            for(uint8_t x=0; x<FIELD_W; x++) field[0][x]=0;
            lines++; y++;          // 같은 행 다시 검사
        }
    }
    // 점수 및 100점 출 트리거
    if(lines){
        uint16_t prev = score;
        score += (uint16_t)lines*100; if(score>999) score=999;
        update_score_digits();
        if((prev/100)<(score/100)) dance_small_timer=DANCE_SMALL_DURATION;
    }
}
```

- 바깥으로 빠져나온 픽셀(스폰 직전 등)은 그냥 버린다. 실제 게임 화면 밖에선 보이지 않기 때문.
- 줄 삭제에서 y++가 중요한 이유: 한 줄을 지우면 그 위 줄들이 내려오는데, 같은 인덱스 y에 새 줄이 오므로 다시 검사해야 다중 라인(더블/트리플/쿼드)이 제대로 카운트된다.
- 점수는 라인×100. 백점 단위가 증가할 때만 “작은 춤” 타이머가 세팅된다(의도적으로 과도한 빈도 억제).

```
// 새 조각 spawn(전장 충돌 시 1 반환)
static uint8_t spawn_piece(void){
    cur_type=rand8()%7; cur_rot=0; cur_x=3; cur_y=0;
    return !can_move(0,0,0); // 못 놓으면 게임오버 조건
}
// 한 틱 진행(자물 낙하+고정+스폰)
static void game_step(void){
    if(stage_clear_active) return; // 축하 줄 정지
    if(game_over) return;
    if(can_move(0,1,0)) cur_y++; // 아래로 한 칸
    else{
        lock_and_clear(); // 고정 후
        if(spawn_piece()) game_over=1; // 새 조각 못 놓으면 종료
    }
}
```

- 시작 x=3은 10칸 폭에서 중앙에 가깝게(왼쪽 여유 3칸).
- 스폰 실패는 곧 게임오버로 즉시 전이. 이후 메인 루프가 GAME/END 모드로 들어간다.

12) 텍스트 오버레이(5×7, 스케일 2)

```
// ----- 텍스트(5x7) 2배 스케일 렌더 -----
// (vy,vx) 위치 픽셀에 문자 겹쳐 그리기
static inline void overlay_text_5x7_s2(uint8_t vy,uint8_t vx,
uint8_t x0,uint8_t y0,
const char* s,uint8_t color,
uint8_t *c_out){
    // 문자 폭 5, 높이 7을 x2 스케일(10x14), 글자 간격 2
    if(vy<y0 || vx<x0) return;
    uint8_t ry = vy - y0;
    uint8_t rx = vx - x0;
    uint8_t char_w = 12; // 10 + 2
    uint8_t idx = rx / char_w; // 몇 번째 글자
    uint8_t inx = rx % char_w; // 글자 내부 x
    if(!s[idx]) return; // 문자열 끝
    if(inx>=10 || ry>=14) return; // 간격/범위 밖
    uint8_t cell_x = inx/2; // 0..4
    uint8_t cell_y = ry/2; // 0..6
    uint8_t bits = pgm_read_byte(&font5x7[fidx(s[idx])][cell_y]);
    if(bits & (1<<(4-cell_x))) *c_out = color;
}
```

- 픽셀 단위로 호출될 때 “지금 픽셀이 글자 영역인지?”만 판단해서 *c_out을 덮어쓴다.
- 스케일 2라서 실제 글자폭 10, 글자 간격 2 → 전체 폭 12. 가독성을 위한 결정.

13) 100점 춤 / 300점 클리어 / GAME-END

```
// ----- 100점: 브물 춤(12x12, 2프레임) -----
static const uint16_t blob1[12] PROGMEM = { /* 등그라미 프레임1 */
    0x03C0,0x07E0,0x0FF0,0x1FF8,0x1FF8,0x1FF8,0x1DF8,0x0FF0,0x07E0,0x03C0,0x0180,0x0180
};
static const uint16_t blob2[12] PROGMEM = { /* 프레임2 약간 이동 */
    0x0180,0x03C0,0x07E0,0x0FF0,0x1DF8,0x1FF8,0x1FF8,0x1FF8,0x0FF0,0x07E0,0x03C0,0x0180
};
static inline void overlay_dance_small(uint8_t vy,uint8_t vx,uint8_t *c,uint16_t tick){
    if(!dance_small_timer) return; // 타이머 없으면 x
    if(vx<DANCE_SMALL_X0 || vx>=DANCE_SMALL_X0+12 || vy<DANCE_SMALL_Y0 || vy>=DANCE_SMALL_Y0+12) return;
    uint8_t px=vx-DANCE_SMALL_X0, py=vy-DANCE_SMALL_Y0;
    const uint16_t* f = ((tick>>2)&1)? blob2:blob1; // 4틱마다 프레임 토글
    uint16_t row = pgm_read_word(&f[py]);
    if(row & (1<<(15-px))) *c = (((tick>>2)&1)? DANCE_SMALL_COL2:DANCE_SMALL_COL1);
}
```

- dance_small_timer가 0이 아니면 우측 영역(44,18)을 12×12 박스로 잡아서 프레임 두 개 (blob1/2)를 번갈아 보인다.

- tick>>2로 속도를 완만하게(4프레임당 토글) → 너무 바쁘지 않고 귀엽게 보이도록.

```
// ----- 300점: STAGE CLEAR(게임 정지+축하) -----
static inline void overlay_stage_clear(uint8_t vy,uint8_t vx,uint8_t *c,uint16_t tick){
    if(!stage_clear_active) return;

    // 회전하는 사선 배경(간단 팔레트)
    uint8_t stripe = ((vx + vy + (tick&31)) & 7);
    const uint8_t pal[4] = { RGB6(3,3,0), RGB6(3,0,3), RGB6(0,3,3), RGB6(3,0,0) };
    *c = pal[stripe & 3];

    // 큰 텍스트 오버레이
    overlay_text_5x7_s2(vy,vx, 6,10, "STAGE", RGB6(3,3,3), c);
    overlay_text_5x7_s2(vy,vx, 6,40, "CLEAR", RGB6(3,3,3), c);
}
```

- 클리어 모드에선 정상 게임 렌더 위가 아니라 전체를 대체한다(뒤에서 return).

- 대각선처럼 보이는 “스트라이프”는 (vx+vy+위상)&7로 만든다. 간단하지만 움직임이 있어 축하감.

```
static inline void overlay_game_end(uint8_t vy,uint8_t vx,uint8_t *c,uint16_t tick){
    if(!game_over || stage_clear_active) return;

    // GAME / END 텍스트
    overlay_text_5x7_s2(vy,vx, 10,10, "GAME", RGB6(3,3,3), c);
    overlay_text_5x7_s2(vy,vx, 16,42, "END", RGB6(3,3,3), c);

    // 가운데 12x12 표정 애니
    uint8_t cx=26, cy=26;
    if(vx>=cx && vx<cx+12 && vy>=cy && vy<cy+12){
        uint8_t px=vx-cx, py=vy-cy;
        const uint16_t* f = ((tick>>3)&1)? sad2:sad1;
        uint16_t row = pgm_read_word(&f[py]);
        if(row & (1<<(15-px))) *c = RGB6(3,0,0);
    }
}
```

- GAME/END 모드는 클리어 모드와 상호 배타. 게임오버 중에는 입력으로만 빠져나간다(아래 메인 참조).

- tick>>3로 클리어 연출보다 더 느리게 깜빡여 “실망” 분위기를 줌.

14) 최종 픽셀 합성(가상 64×64 → R/G/B 2비트)

```
// ----- 픽셀 계산 -----
// (vy,vx)의 최종 RGB(2bit씩) 계산
static inline void get_pixel_color_virt(uint8_t vy,uint8_t vx,
uint8_t *r,uint8_t *g,uint8_t *b,
uint16_t tick)
{
    // 기본 바닥판 배경
    uint8_t c = (((vx>>1)+(vy>>1))&1)? COL_BG1:COL_BG2;

    // 스테이지 클리어면 전체를 축하 오버레이로 대체
    if(stage_clear_active){
        overlay_stage_clear(vy,vx,&c,tick);
        *r=c&3; *g=(c>>2)&3; *b=(c>>4)&3; return;
    }

    // 외곽 테두리(밝기 두 단계)
    uint8_t pw = FIELD_W*BLOCK_SZ;
    uint8_t ix0=PLAY_X0, ix1=PLAY_X0+pw-1;
    uint8_t bx0 = (ix0>=2)? ix0-2:0, bx1=ix1+2;
    if((vx==bx0||vx==bx0+1||vx==bx1||vx==bx1-1))
    c=((vx==bx0||vx==bx1)? COL_BORDER_BRT:COL_BORDER_DIM);
    if(vy==0||vy==VIRT_H-1){
        if(vx==bx0&&vx<=bx1) c=((vy==0)?COL_BORDER_BRT:COL_BORDER_DIM);
    }

    // 플레이 필드(고정블록 + 현재조각 합성)
    if(vx>=ix0 && vx<ix0+pw){
        uint8_t col=(vx-ix0)>>1; // 2픽셀=1칸
        uint8_t cy = vy>>1;
        if(cy<FIELD_H){
            uint8_t t=field[cy][col]; // 기본은 고정 블록
            // 현재 조각이 덮는 영역이면 미리 그려서 미리보기
            if((int16_t)col>=(int16_t)cur_x && (int16_t)col<(int16_t)(cur_x+4) &&
            (int16_t)cy>=(int16_t)cur_y && (int16_t)cy<(int16_t)(cur_y+4)){
                int16_t lx=(int16_t)col-(int16_t)cur_x;
                int16_t ly=(int16_t)cy -(int16_t)cur_y;
                if(lx==0&&lx<4&&ly==0&&ly<4){
                    uint8_t row=pgm_read_byte(&tetromino[cur_type][cur_rot][(uint8_t)ly]);
                    if(row&(1<<(3-(uint8_t)lx))) t=cur_type+1;
                }
            }
            if(t) c=block_color(t);
            else c=((col+cy)&1==0)? RGB6(0,0,1):RGB6(0,0,0); // 바닥 무늬
        }
    }

    // 점수 박스 그리기(테두리+배경)
    if(vx>=SCORE_X0-2 && vx<=SCORE_X0+18 && vy>=SCORE_Y0-2 && vy<=SCORE_Y0+10){
        if(vx==SCORE_X0-2||vx==SCORE_X0+18||vy==SCORE_Y0-2||vy==SCORE_Y0+10) c=COL_BORDER_BRT;
        else if(vx==SCORE_X0-1||vx==SCORE_X0+17||vy==SCORE_Y0-1||vy==SCORE_Y0+9) c=COL_BORDER_DIM;
        else c=COL_SCOREBOX;
    }

    // 점수 숫자(빨강)
    if(vx>=SCORE_X0 && vx<SCORE_X0+12 && vy>=SCORE_Y0 && vy<SCORE_Y0+5){
        uint8_t dx=vx-SCORE_X0, di=dx/4, px=dx%4;
        if(di<3 && px<3){
            uint8_t dy=vy-SCORE_Y0;
            uint8_t d=score_dig[di];
            uint8_t bits=pgm_read_byte(&digit_font[d][dy]);
            if(bits & (1<<(2-px))) c=RGB6(3,0,0);
        }
    }

    // 100점 보름 줄
    overlay_dance_small(vy,vx,&c,tick);
    // GAME/END 애니
    overlay_game_end(vy,vx,&c,tick);

    // 최종 색을 2비트 RGB로 조개기
    *r=c&3; *g=(c>>2)&3; *b=(c>>4)&3;
}
```

- 레이어 우선순위 = 코드 호출 순서. 나중에 호출된 오버레이가 앞에 덮인다.
- “현재 조각 미리 합성”을 통해 고정되기 전에 위치를 보여준다(유저 피드백).
- 점수는 12px 폭(3자리 × 각 4px 폭). 각 자리 내부에서 3×5 비트마스크로 픽셀을 켜다.

15) HUB75 스캔(행쌍 + 2비트 PWM)

```
// ----- HUB75 스캔(두 패널) -----
static inline void set_row(uint8_t row){           // A-D 라인 선택
    uint8_t v=PORTC&0xF0; v|=(row&0x0F); PORTC=v;
}
```

- HUB75는 A,B,C,D로 0~15 선택 → 실제 패널은 “상·하 행쌍”이 함께 구동된다.

```
static inline void shift_rowpair(uint8_t rowpair,uint8_t bit,uint16_t tick){
    for(uint8_t x=0;x<SHIFT_W;x++){                // 한 줄 시프트 출력
        uint8_t r1,g1,b1,r2,g2,b2;
        uint8_t chain=(x>=PANEL_W)?1:0;           // 체인 선택
        uint8_t vx=(x%PANEL_W);
        uint8_t vy_top    = rowpair + (chain?32:0); // 위쪽 행의 가상 y
        uint8_t vy_bottom = rowpair + (chain?48:16); // 아래쪽 행의 가상 y
        get_pixel_color_virt(vy_top, vx,&r1,&g1,&b1,tick);
        get_pixel_color_virt(vy_bottom,vx,&r2,&g2,&b2,tick);

        uint8_t v=PORTD&0x03;                      // 상·하 RGB 비트 설정
        v&=~((1<<R1_PD)|(1<<G1_PD)|(1<<B1_PD)|(1<<R2_PD)|(1<<G2_PD)|(1<<B2_PD));
        if((r1>>bit)&1) v|=(1<<R1_PD);
        if((g1>>bit)&1) v|=(1<<G1_PD);
        if((b1>>bit)&1) v|=(1<<B1_PD);
        if((r2>>bit)&1) v|=(1<<R2_PD);
        if((g2>>bit)&1) v|=(1<<G2_PD);
        if((b2>>bit)&1) v|=(1<<B2_PD);
        PORTD=v;
        CLK_HIGH(); _delay_us(1); CLK_LOW();        // 한 픽셀 시프트
    }
}
```

- bit=0/1 두 패스로 돌며 LSB, MSB 순서 PWM 구현.
- vy_top/vy_bottom 계산이 포인트: 체인된 두 패널의 “위/아래 절반”이 가상 64×64 어디에 대응되는지 정확히 잡아준다(0~15, 16~31, 32~47, 48~63).

```
static void scan_frame(uint16_t tick){              // 프레임 전체 스캔
    const uint8_t base_T_us=40;                    // PWM 기본 ON 시간
    for(uint8_t bit=0;bit<2;bit++){                // 2비트 PWM(4레벨)
        uint8_t on=(1<<bit)*base_T_us;             // 가중치
        for(uint8_t row=0;row<16;row++){            // HUB75는 상·하 쌍 행
            set_row(row);
            shift_rowpair(row,bit,tick);
            LAT_HIGH(); _delay_us(1); LAT_LOW();    // 래치
            OE_LOW(); for(uint8_t t=0;t<on;t++) _delay_us(1); OE_HIGH(); // 발광 시간
        }
    }
}
```

- 한 프레임 = bit(2) × row(16) 묶음.
- OE ON 시간을 비트 가중치로 배분(0비트=40 μs, 1비트=80 μs → 총 120 μs/행쌍).
- 전체 프레임 시간 ≈ 16×(120 μs + 시프트/래치 오버헤드)×(2비트). 패널에 따라 base_T_us 조절로 밝기/플리커 균형 맞춘다.

16) 입력 오토리핏(에지+자동반복)

```
// ----- 입력(오토리핏) -----
typedef struct{ uint8_t prev,hold; } BtnState; // 에지/홀드 상태
static BtnState stL={0,0},stR={0,0},stD={0,0},stRot={0,0};
#define REPEAT_DELAY 6 // 초지연
#define REPEAT_RATE 2 // 반복 속도
static uint8_t edge_or_rep(uint8_t now, BtnState* st){
    uint8_t fire=0;
    if(now){
        if(!st->prev){ fire=1; st->hold=0; } // 눌린 에지
        else{
            st->hold++;
            if(st->hold>REPEAT_DELAY && ((st->hold-REPEAT_DELAY)%REPEAT_RATE==0)) fire=1; // 자동반복
        }
    }else st->hold=0;
    st->prev=now; return fire;
}
```

- 키를 꼭 누르면 REPEAT_DELAY(6틱) 후 REPEAT_RATE(2틱 간격)로 반복 이벤트를 발생.
- 좌우 이동/회전/소프트드롭 각각 독립 상태를 갖는다(문제 상황: 동시 입력 시 가속 비대칭 방지).

17) 속도 곡선(점수→낙하 간격)

```
// ----- 메인 -----
static void update_speed_from_score(void){ // 점수→낙하속도
    int16_t t=18-(int16_t)(score/50);
    if(t<DROP_FRAMES_MIN) t=DROP_FRAMES_MIN;
    drop_frames=(uint8_t)t;
}
```

- 50점마다 1프레임씩 빨라진다(18→...→최소 8).
- 너무 빨라지지 않게 하한(DROP_FRAMES_MIN)을 둔다. 유저 체감이 부드럽게 올라가도록 선형.

18) 메인 루프(상태 전이 + 렌더/게임 구동)

```
int main(void){
    // 포트 방향 및 플립 설정
    DORD |= (1<<R1_PD)|(1<<G1_PD)|(1<<B1_PD)|(1<<R2_PD)|(1<<G2_PD)|(1<<B2_PD);
    DORB |= (1<<CLK_PB)|(1<<OE_PB)|(1<<LAT_PB);
    DORC |= (1<<A_PC)|(1<<B_PC)|(1<<C_PC)|(1<<D_PC);
    DORB&=~((1<<BTN_LEFT_PB)|(1<<BTN_DOWN_PB)); PORTB |= ((1<<BTN_LEFT_PB)|(1<<BTN_DOWN_PB));
    DORC&=~((1<<BTN_RIGHT_PC)|(1<<BTN_ROT_PC)); PORTC |= ((1<<BTN_RIGHT_PC)|(1<<BTN_ROT_PC));
    CLK_LOW(); LAT_LOW(); DE_HIGH(); // 초기 신호 상태

    uint16_t tick=0; // 전역 틱
    clear_field(); // 필드 초기화
    if(spawn_piece()) game_over=1; // 시작 조각 배치 실패시 종료

    while(1){
        scan_frame(tick); tick++; // 한 프레임 출력

        // 300점: 스테이지 풀리어 시작(한 번만)
        if(!stage_clear_active && score>=300){
            stage_clear_active=1;
            stage_clear_timer = STAGE_CLEAR_DURATION;
        }

        if(stage_clear_active){
            if(stage_clear_timer) stage_clear_timer--; // 카운트 다운
            // 끝나면 아무 버튼으로 재시작
            if(!stage_clear_timer){
                if(btn_left()||btn_right()||btn_down()||btn_rot()){
                    clear_field(); update_speed_from_score();
                    if(spawn_piece()) game_over=1;
                }
            }
            continue; // 게임 로직 정지(연출만)
        }

        if(dance_small_timer) dance_small_timer--; // 100점 춤 타이머

        // 입력 샘플링
        uint8_t l=btn_left(), r=btn_right(), d=btn_down(), rt=btn_rot();

        // 게임오버 화면: 아무 키로 재시작
        if(game_over){
            if(l||r||d||rt){
                clear_field(); update_speed_from_score();
                if(spawn_piece()) game_over=1;
            }
            continue; // GAME/END 유지
        }

        // 좌/우/회전/소프트드롭(오토리밋 적용)
        if(edge_or_rep(l,&stL)){ if(can_move(-1,0,0)) cur_x--; }
        if(edge_or_rep(r,&stR)){ if(can_move(+1,0,0)) cur_x++; }
        if(edge_or_rep(rt,&stRot)){ if(can_move(0,0,+1)) cur_rot=(cur_rot+1)&3; }
        if(edge_or_rep(d,&stD)){
            for(uint8_t i=0;i<SOFT_DROP_STEPS;i++){ if(can_move(0,+1,0)) cur_y++; else break; }
        }

        // 자동 낙하 타이밍
        frame_cnt++;
        if(frame_cnt>=drop_frames){
            frame_cnt=0;
            game_step(); // 한 칸 내리거나 고정
            if(!game_over){
                if(!can_move(0,0,0)){ if(spawn_piece()) game_over=1; } // 천장 체크
            }
        }
    }
    return 0;
}
```

- 상위 가드: stage_clear_active → game_over → 정상 플레이. 위에서부터 체크해서 “연출 모드”나 “오버 모드”일 땐 게임 로직을 아예 건드리지 않음.
- 재시작 입력: 두 모드 모두 “아무 버튼”으로 clear_field() → 속도 갱신 → 스폰.
- 입력 처리 순서: 좌/우 → 회전 → 소프트드롭. 충돌을 매번 체크해 “벽 관통” 방지.

- 자동 낙하는 입력 처리 이후 수행. 이유: 같은 프레임에 좌/우/회전하고 곧바로 중력이 적용되어도 사용감이 자연스럽다.
- 희귀 케이스 보정: game_step() 뒤에 !can_move(0,0,0)를 다시 본다. 아주 타이트한 상황 (스폰 직후 상단 겹침 등)에서 즉시 오버로 전이시키기 위함.

3. 전체 회로도

